



Singularry Launchpad

AUDIT REPORT

Version 1.0.0

Serial No. 2026021000012016

Presented by Fairyproof

February 10, 2026

01. Introduction

This document includes the results of the audit performed by the Fairyproof team on the Singularry launchpad project.

Audit Start Time:

January 20, 2026

Audit End Time:

February 9, 2026

Audited Code's Github Repository:

<https://github.com/singularry/sly-launchpad>

Audited Code's Github Commit Number When Audit Started:

396681083242f2d735ecb970c6add9c5b5a35882

Audited Code's Github Commit Number When Audit Ended:

91c946929a56c03136d6421eae9ab237d5e54dc0

Audited Source Files:

The source files audited include all the files as follows:

```
contracts
├── BondingCurveV4.sol
├── Factory.sol
├── FeeCollector.sol
├── LockV2.sol
├── TokenImplementation.sol
├── gradulators
│   ├── PancakeSwapGraduatorV2.sol
│   └── ThenaGraduatorV2.sol
├── interfaces
│   ├── IBondingCurve.sol
│   ├── IDEXGraduator.sol
│   ├── IFactory.sol
│   ├── ILock.sol
│   ├── IToken.sol
│   ├── UniswapV3Interfaces.sol
│   └── algebra
│       ├── IAlgebraFactory.sol
│       ├── IAlgebraPool.sol
│       ├── IAlgebraPoolDeployer.sol
│       └── INonfungiblePositionManager.sol
└── libraries
    └── BondingMathV2.sol
```

5 directories, 18 files

The goal of this audit is to review Singularity's solidity implementation for its launchpad function, study potential security vulnerabilities, its general design and architecture, and uncover bugs that could compromise the software in production.

We make observations on specific areas of the code that present concrete problems, as well as general observations that traverse the entire codebase horizontally, which could improve its quality as a whole.

This audit only applies to the specified code, software or any materials supplied by the Singularity team for specified versions. Whenever the code, software, materials, settings, environment etc is changed, the comments of this audit will no longer apply.

— Disclaimer

Note that as of the date of publishing, the contents of this report reflect the current understanding of known security patterns and state of the art regarding system security. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. If the audited source files are smart contract files, risks or issues introduced by using data feeds from offchain sources are not extended by this review either.

Given the size of the project, the findings detailed here are not to be considered exhaustive, and further testing and audit is recommended after the issues covered are fixed.

To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

— Methodology

The above files' code was studied in detail in order to acquire a clear impression of how the its specifications were implemented. The codebase was then subject to deep analysis and scrutiny, resulting in a series of observations. The problems and their potential solutions are discussed in this document and, whenever possible, we identify common sources for such problems and comment on them as well.

The Fairyproof auditing process follows a routine series of steps:

1. Code Review, Including:

- Project Diagnosis

Understanding the size, scope and functionality of your project's source code based on the specifications, sources, and instructions provided to Fairyproof.

- Manual Code Review

Reading your source code line-by-line to identify potential vulnerabilities.

- Specification Comparison

Determining whether your project's code successfully and efficiently accomplishes or executes its functions according to the specifications, sources, and instructions provided to Fairyproof.

2. Testing and Automated Analysis, Including:

- Test Coverage Analysis

Determining whether the test cases cover your code and how much of your code is exercised or executed when test cases are run.

- Symbolic Execution

Analyzing a program to determine the specific input that causes different parts of a program to execute its functions.

3. Best Practices Review

Reviewing the source code to improve maintainability, security, and control based on the latest established industry and academic practices, recommendations, and research.

— Structure of the document

This report contains a list of issues and comments on all the above source files. Each issue is assigned a severity level based on the potential impact of the issue and recommendations to fix it, if applicable. For ease of navigation, an index by topic and another by severity are both provided at the beginning of the report.

— Documentation

For this audit, we used the following source(s) of truth about how the token issuance function should work:

Website: <https://singularray.org/>

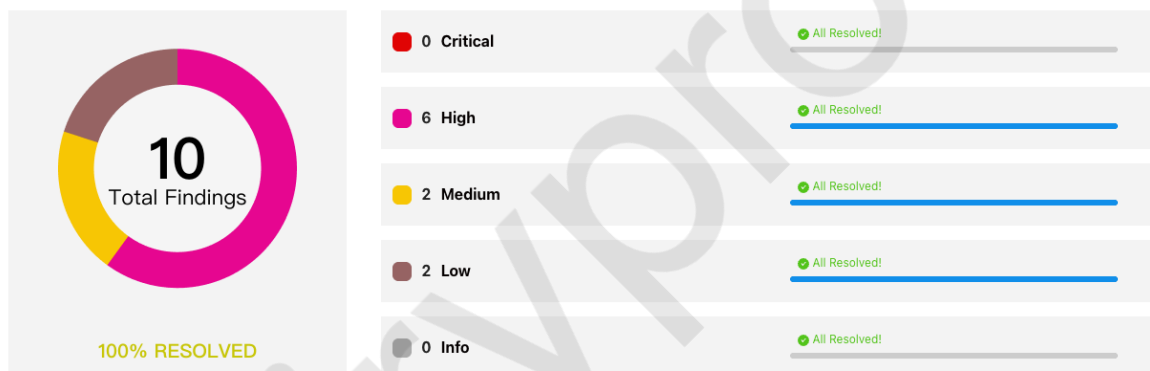
Whitepaper: <https://docs.singularray.org/>

Source Code: <https://github.com/singularray/sly-launchpad>

These were considered the specification, and when discrepancies arose with the actual code behavior, we consulted with the Singularray team or reported an issue.

— Comments from Auditor

Serial Number	Auditor	Audit Time	Result
2026021000012016	Fairyproof Security Team	Jan 20, 2026 - Feb 9, 2026	Passed



Summary:

The Fairyproof security team used its auto analysis tools and manual work to audit the project. During the audit, six issues of high-severity, two issues of medium-severity and two issues of low-severity were uncovered. The Singularray team fixed six issues of high, two issues of medium and two issues of low, and acknowledged the remaining issues.

02. About Fairyproof

[Fairyproof](#) is a leading technology firm in the blockchain industry, providing consulting and security audits for organizations. Fairyproof has developed industry security standards for designing and deploying blockchain applications.

03. Introduction to Singularity

Singularity is a platform of creating next-generation financial systems with intelligent, autonomous protocols and seamless cross-chain operations.

The above description is quoted from relevant documents of Singularity.

04. Major functions of audited code

The audited code implements a token launch and presale platform designed for BNB Chain. It provides a fully on-chain `Launchpad` where new tokens are created, sold via a bonding-curve based sale, and then automatically graduated to concentrated-liquidity DEX pools (PancakeSwap V3 and ThenaSwap V3). The platform aims to standardize the lifecycle of a new token: deployment, initial price discovery and distribution, DEX listing, and long-term liquidity/fee management.

During the bonding curve phase, users buy and sell the token against a virtual/real reserve model, with configurable fees that are split between the token creator and the platform. Once certain economic conditions are met, the bonding curve finalizes and the remaining reserves are used to create a full-range LP position on supported DEXes. The resulting LP NFT is permanently locked in a dedicated lock contract, and a `FeeCollector` contract tracks and distributes ongoing DEX trading fees between the creator and the platform.

Overall, the system is intended to give token creators a safer, more transparent launch flow (predefined supply split, automatic liquidity provisioning, enforced LP lock) while giving participants clear, on-chain rules for pricing, fees, and post-launch liquidity behavior.

05. Coverage of issues

The issues that the Fairyproof team covered when conducting the audit include but are not limited to the following ones:

- Access Control
- Admin Rights
- Arithmetic Precision
- Code Improvement
- Contract Upgrade/Migration
- Delete Trap
- Design Vulnerability
- DoS Attack
- EOA Call Trap

- Fake Deposit
- Function Visibility
- Gas Consumption
- Implementation Vulnerability
- Inappropriate Callback Function
- Injection Attack
- Integer Overflow/Underflow
- IsContract Trap
- Miner's Advantage
- Misc
- Price Manipulation
- Proxy selector clashing
- Pseudo Random Number
- Re-entrancy Attack
- Replay Attack
- Rollback Attack
- Shadow Variable
- Slot Conflict
- Token Issuance
- Tx.origin Authentication
- Uninitialized Storage Pointer

06. Severity level reference

Every issue in this report was assigned a severity level from the following:

Critical severity issues need to be fixed as soon as possible.

High severity issues will probably bring problems and should be fixed.

Medium severity issues could potentially bring problems and should eventually be fixed.

Low severity issues are minor details and warnings that can remain unfixed but would be better fixed at some point in the future.

Informational is not an issue or risk but a suggestion for code improvement.

07. Major areas that need attention

Based on the provided source code the Fairyproof team focused on the possible issues and risks related to the following functions or areas.

- Function Implementation

We checked whether or not the functions were correctly implemented.

We found some issues, for more details please refer to [FP-1,FP-2,FP-4,FP-5,FP-6,FP-7,FP-8,FP-10] in "09. Issue description".

- Access Control

We checked each of the functions that could modify a state, especially those functions that could only be accessed by owner or administrator

We didn't find issues or risks in these functions or areas at the time of writing.

- Token Issuance & Transfer

We examined token issuance and transfers for situations that could harm the interests of holders.

We found one issue, for more details please refer to [FP-3] in "09. Issue description".

- State Update

We checked some key state variables which should only be set at initialization.

We didn't find issues or risks in these functions or areas at the time of writing.

- Asset Security

We checked whether or not all the functions that transfer assets were safely handled.

We didn't find issues or risks in these functions or areas at the time of writing.

- Miscellaneous

We checked the code for optimization and robustness.

We found one issue, for more details please refer to [FP-9] in "09. Issue description".

08. List of issues by severity

Index	Title	Issue/Risk	Severity	Status
FP-1	Auto-buy via receive() misroutes tokens and can cause BNB loss	Design Vulnerability	High	✓ Fixed
FP-2	Graduation relies on unconstrained DEX graduator choice	Design Vulnerability	High	✓ Fixed
FP-3	Using Uniform Fixed Parameters for Token Deployment	Design Vulnerability	High	✓ Fixed
FP-4	Incorrect Use of renounceOwnership	Code Improvement	High	✓ Fixed
FP-5	Computation of <code>tickLower</code> and <code>tickUpper</code> is incorrect	Design Vulnerability	High	✓ Fixed
FP-6	Not Converting BNB to WBNB and Not Distinguishing token0/token1	Design Vulnerability	High	✓ Fixed
FP-7	Significant Token Price Discrepancies Across Different Sale Phases	Design Vulnerability	Medium	✓ Fixed
FP-8	Unclear Design Intent of the Lock Contract	Design Vulnerability	Medium	✓ Fixed
FP-9	Overlap Between Foundry and Factory Functionality	Code Improvement	Low	✓ Fixed
FP-10	Possible tokenId Collision	Design Vulnerability	Low	✓ Fixed

09. Issue descriptions

[FP-1] Auto-buy via receive() misroutes tokens and can cause BNB loss

Design Vulnerability

High

✓ Fixed

Issue/Risk: Design Vulnerability

Description:

`BondingCurveV3` implements a `receive()` function that, during the Bonding phase, automatically calls `this.buyTokens{value: msg.value}(0)` when receiving plain BNB above `minContribution`, but this is an external self-call, inside `buyTokens` the `msg.sender` is the bonding curve contract itself, not the original user who sent BNB. So `buyTokens` transfers purchased tokens to the curve, not to the user. And the call hard-codes `minTokens = 0`, so there is no slippage protection even if the recipient logic were correct.

Recommendation:

Remove the auto-buy logic from `receive()` entirely.

Update:

The logic previously implemented in the receive function has been removed, and access has been restricted so that only the WBNB or the Graduator contract can invoke it.

Status:

The team has fixed the issue.

[FP-2] Graduation relies on unconstrained DEX graduator choice

Design Vulnerability

High

✓ Fixed

Issue/Risk: Design Vulnerability

Description:

Under the current design, the creator is able to deploy the BondingCurve contract without selecting a graduator, and assign it later. This creates a centralized dependency, as an unset or incorrectly configured graduator could result in unintended or unexpected behavior.

Recommendation:

Require a supported graduator to be specified during the deployment of the BondingCurve contract.

Update:

The ability to reset the graduator has been removed from the BondingCurve contract, and the graduator parameter is now validated during BondingCurve creation in the Factory contract.

Status:

The team has fixed the issue.

[FP-3] Using Uniform Fixed Parameters for Token Deployment

Design Vulnerability

High

✓ Fixed

Issue/Risk: Design Vulnerability

Description:

In the Factory contract, once it is initialized (a new Factory is created via clone and then initialized), its settings parameters are fixed. Any token deployed through this Factory's `deployBondingCurveSystem` function will always use the same BondingCurve parameters, which means all token economics are identical. This design likely needs further discussion.

Although the contract provides an `updateBondingCurveSettings` function, it is restricted to `onlyOwner` , and there is no way to adjust parameters per token right before creation.

Recommendation:

Pass some parameters (for example `virtualEth` and `bondingTarget`) directly as arguments to the `deployBondingCurveSystem` function, instead of always using the same fixed parameters.

Update:

Parameters of `virtualEth` and `bondingTarget` have been added in the `deployBondingCurveSystemAdvanced` function.

Status:

The team has fixed the issue.

[FP-4] Incorrect Use of `renounceOwnership`

Code Improvement

High

✓ Fixed

Issue/Risk: Code Improvement

Description:

In the `_finalizeCurve` function, there is a call to `renounceOwnership` , which requires the correct privileges to execute, current the `msg.sender` can be user. So the current call will fail.

Recommendation:

Replace `renounceOwnership` with `_transferOwnership(address(0))` .

Update:

Corrections were made using the recommended option.

Status:

The team has fixed the issue.

[FP-5] Computation of `tickLower` and `tickUpper` is incorrect

Design Vulnerability

High

✓ Fixed

Issue/Risk: Design Vulnerability

Description:

In `PancakeSwapGraduatorV2` , the computation of `tickLower` and `tickUpper` is incorrect because it does not take `tickSpacing` multiples into account.

Recommendation:

```

interface IUniswapV3Factory {
function createPool(
address tokenA,
address tokenB,
uint24 fee
) external returns (address pool);
function getPool(
address tokenA,
address tokenB,
uint24 fee
) external view returns (address pool);
// added
function feeAmountTickSpacing(
uint24 fee
) external pure returns (int24 tickSpacing);
}
// Derive full-range ticks aligned to pool's tickSpacing to avoid
_modifyPosition reverts
int24 tickSpacing = factory.feeAmountTickSpacing(DEFAULT_FEE);
int24 tickLower = (MIN_TICK / tickSpacing) * tickSpacing;
int24 tickUpper = (MAX_TICK / tickSpacing) * tickSpacing;

```

Update:

Corrections were made using the recommended option.

Status:

The team has fixed the issue.

[FP-6] Not Converting BNB to WBNB and Not Distinguishing token0/token1

Design Vulnerability

High

✓ Fixed

Issue/Risk: Design Vulnerability

Description:

In the `FeeCollector` contract's `registerGraduatedToken` function, when `preGradFees > 0`, the corresponding BNB should be converted to WBNB; otherwise, the contract's on-chain balance and the recorded values will not match. Additionally, when calculating `creatorAccumulated0 / creatorAccumulated1`, it does not distinguish whether WBNB is `token0` or `token1`. WBNB in the pool might be `token0` or `token1`.

Recommendation:

```

address internal constant WBNB_ADDRESS =
0xbb4CdB9CBd36B01bD1cBaEBF2De08d9173bc095c;
bool flag = token < WBNB_ADDRESS;
if (preGradFees > 0) {

```

```

    (bool success, ) = payable(WBNB_ADDRESS).call{value: preGradFees}("");
    require(success, "wnnb deposit failed");
    creatorPreGrad = (preGradFees * creatorFeeBps) / 10000;
    platformPreGrad = preGradFees - creatorPreGrad;
}
graduatedTokens[token] = GraduatedToken({
    token: token,
    creator: creator,
    lockContract: lockContract,
    positionManager: positionManager,
    lpTokenId: lpTokenId,
    pool: pool,
    creatorAccumulated0: flag ? 0 : creatorPreGrad, // Pre-grad fees in token
    creatorAccumulated1: flag ? creatorPreGrad : 0, // Pre-grad fees in
    native token
    platformAccumulated0: flag ? 0 : platformPreGrad,
    platformAccumulated1: platformPreGrad : 0,
    totalClaimed0: flag ? 0 : preGradFees,
    totalClaimed1: flag ? preGradFees : 0,
    lastClaimTimestamp: block.timestamp,
    isRegistered: true
});

```

Update:

Corrections were made using the recommended option.

Status:

The team has fixed the issue.

[FP-7] Significant Token Price Discrepancies Across Different Sale Phases

Design Vulnerability

Medium

✓ Fixed

Issue/Risk: Design Vulnerability

Description:

The token sale mechanism is divided into PreBonding, Bonding, and Finalized phases. A fixed linear pricing model is used during the Bonding phase, after which pricing transitions to an AMM-based model. Upon reaching the sale target, the token is finalized and traded on a live DEX. Because the design relies on a single-sided virtual reserve, substantial price discrepancies may occur between the different phases, potentially resulting in unexpected price jumps.

Recommendation:

Adjust the design to use a dual-sided virtual reserve so that token prices transition more smoothly between different sale phases.

Update:

The project team updated the design by referencing the Pump model and adopting a Dual Virtual Reserve System.

Status:

The team has fixed the issue.

[FP-8] Unclear Design Intent of the Lock Contract

Design Vulnerability

Medium

✓ Fixed

Issue/Risk: Design Vulnerability

Description:

The current Lock contract is designed to be deployed using a clone-based pattern. However, the contract does not inherit from a base contract that supports initialization via an initializer function. In addition, some parameters are initialized in the constructor rather than in an initialization function, resulting in an inconsistent and potentially confusing initialization flow.

Recommendation:

Given the actual business requirements, the Lock contract does not need to adopt a clone-based or proxy/implementation pattern. Instead, it is recommended to deploy the contract as a standalone instance and perform all necessary initialization directly within the constructor, removing the associated clone-related design.

Update:

The project team removed the clone-related design from the Lock contract

Status:

The team has fixed the issue.

[FP-9] Overlap Between Foundry and Factory Functionality

Code Improvement

Low

✓ Fixed

Issue/Risk: Code Improvement

Description:

In the launchpad design, the platform uses the Foundry contract to deploy the `Factory` and `LockV2` contracts, and then token project owners use the Factory to deploy tokens and the `BondingCurveV3` contract. The current design allows Foundry to deploy multiple Factory/LockV2 instances (using the clone pattern), but in practice only one Factory will be used.

Recommendation:

Remove the Foundry contract, and optionally modify Factory and LockV2 to be non-template (non-clone) contracts.

Update:

Corrections were made using the recommended option.

Status:

The team has fixed the issue.

[FP-10] Possible tokenId Collision

Design Vulnerability

Low

✓ Fixed

Issue/Risk: Design Vulnerability

Description:

Current Lock contract use `mapping(uint256 => LockedNFT) public lockedNFTs;` to determine whether a given token has provided LP and been locked. However, different DEXs may generate the same lpTokenId (each starts from 1 and increments). In that case, tokens with a duplicated `lpTokenId` would not be able to be locked.

Recommendation:

Deploy multi separate `Lock` contracts, each dedicated to one `Graduator` and `Dex`.

Update:

Added `lockContract` state variable in the `Graduator` contract to record the binding `Lock` contract.

Status:

The team has fixed the issue.

10. Recommendations to enhance the overall security

We list some recommendations in this section. They are not mandatory but will enhance the overall security of the system if they are adopted.

- Consider managing the owner's access control with great care and transferring it to a multi-sig wallet or DAO when necessary.

11. Appendices

11.1 Unit Test

1. LaunchpadPancakeForkingTest.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import {Test} from "forge-std/Test.sol";
import {ERC1967Proxy} from
"@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {Factory,IFactory,OwnableUpgradeable} from
"../../src/launchpad/Factory.sol";
import {TokenImplementation} from
"../../src/launchpad/TokenImplementation.sol";
import {BondingCurveV4} from " ../../src/launchpad/BondingCurveV4.sol";
import {LockV2,ILock} from " ../../src/launchpad/LockV2.sol";
import {PancakeSwapGraduatorV2} from
" ../../src/launchpad/graduators/PancakeSwapGraduatorV2.sol";
import {FeeCollector} from " ../../src/launchpad/FeeCollector.sol";

import {console} from "forge-std/console.sol";

interface ISwapRouter {
    struct ExactInputSingleParams {
        address tokenIn;
        address tokenOut;
        uint24 fee;
        address recipient;
        uint256 deadline;
        uint256 amountIn;
        uint256 amountOutMinimum;
        uint160 sqrtPriceLimitX96;
    }

    /// @notice Swaps `amountIn` of one token for as much as possible of
    another token
    /// @param params The parameters necessary for the swap, encoded as
    `ExactInputSingleParams` in calldata
    /// @return amountOut The amount of the received token
    function exactInputSingle(ExactInputSingleParams calldata params) external
    payable returns (uint256 amountOut);
}

contract LaunchpadPancakeForkingTest is Test {
    uint256 internal constant factoryFees = 0.1 ether;
```



```

    address internal constant PANCAKESWAP_POSITION_MANAGER =
0x46A15B0b27311cedF172AB29E4f4766fbE7F4364;
    address internal constant THENA_POSITION_MANAGER =
0x643B68Bf3f855B8475C0A700b6D1020bfc21d02e;
    address internal constant PANCAKESWAP_FACTORY =
0x0BFbCF9fa4f9C56B0F40a671Ad40E0805A091865;
    address internal constant PANCAKE_V3_SWAP_ROUTER =
0x1b81D678ffb9C0263b24A97847620C99d213eB14;
    address internal constant WBNB =
0xbb4CdB9CBd36B01bD1cBaEBF2De08d9173bc095c;
    uint16 internal constant BONDING_ALLOCATION_BPS = 8000; // 80% for
bonding curve
    uint16 internal constant DEX_ALLOCATION_BPS = 2000; // 20% for DEX
liquidity
    uint16 internal constant VIRTUAL_TOKEN_MULTIPLIER_BPS = 13500;

    address internal owner = makeAddr("owner");
    address internal feeTo = makeAddr("feeTo");
    address internal creator = makeAddr("creator");
    address internal alice = address(0x1234);
    address internal bob = address(0x5678);

    Factory internal factory;
    LockV2 internal lock;
    TokenImplementation internal token_impl;
    BondingCurveV4 internal bonding_impl;
    PancakeSwapGraduatorV2 internal graduator;
    FeeCollector internal feeCollector;
    ISwapRouter internal router = ISwapRouter(PANCAKE_V3_SWAP_ROUTER);

    function setUp() public {
        vm.createSelectFork("your endpoint here",79555399);
        token_impl = new TokenImplementation();
        bonding_impl = new BondingCurveV4();
        Factory impl = new Factory();
        lock = new LockV2(owner, PANCAKESWAP_POSITION_MANAGER,
THENA_POSITION_MANAGER);
        IFactory.BondingCurveSettings memory defaultSettings =
IFactory.BondingCurveSettings({
            virtualEth: 20 ether,
            bondingTarget: 70 ether,
            minContribution: 0.01 ether,
            buyFee: 100,
            sellFee: 100,
            creatorFeeBps: 5000,
            weth: WBNB,
            feeTo: feeTo
        });
        ERC1967Proxy proxy = new ERC1967Proxy(
            address(impl),
            abi.encodeWithSelector(
                Factory.initialize.selector,
                factoryFees,
                owner,

```

```

        token_impl,
        bonding_impl,
        lock,
        defaultSettings
    )
);
factory = Factory(payable(address(proxy)));

vm.startPrank(owner);
feeCollector = new FeeCollector(feeTo,WBNB);
graduater = new PancakeSwapGraduatorV2(
    address(factory),
    PANCAKESWAP_FACTORY,
    PANCAKESWAP_POSITION_MANAGER
);
// Related settings, quite complex
graduater.setFeeCollector(address(feeCollector));
graduater.setLockContract(address(lock));
factory.setGraduatorWhitelisted(address(graduator),true);
lock.setAuthorizedFeeClaimer(address(feeCollector),true);
feeCollector.setAuthorizedRegistrar(address(graduator),true);
vm.stopPrank();
}

function testBuyAndSellToken() public {
    // Additional test logic can be implemented here to test the forking
    setup
    // 1. DeployBondingCurveSystem
    vm.deal(creator, 1 ether);
    uint256 initialBuyAmount = 0.1 ether;
    vm.prank(creator);
    (address token,address bonding) =
    factory.deployBondingCurveSystemAdvanced{value: initialBuyAmount +
    factoryFees}(
        "TestToken",
        "TTK",
        initialBuyAmount,
        address(graduator),
        1 ether,    // virtualEth
        1 ether    // bondingTarget
    );
    // 2. Verify deployment
    // Check deployment fees
    assertEq(address(factory).balance, factoryFees); // 0.1 ether
    deployment fees

    BondingCurveV4 bondingCurve = BondingCurveV4(payable(bonding));
    // Check total token supply
    uint256 totalSupply = ERC20(token).totalSupply();
    assertEq(totalSupply, 1_000_000_000 ether); // Total supply should be
    1,000,000,000 TTK

    // Check token allocation

```

```

uint256 bondingAllocation = (totalSupply * BONDING_ALLOCATION_BPS) /
10000;
assertEq(bondingCurve.bondingAllocation(), bondingAllocation);
assertEq(bondingCurve.dexAllocation(), totalSupply -
bondingAllocation);

// Check reserves
uint256 fee = initialBuyAmount * 100 / 10000; // 1% buyFee
console.log("Initial Buy Fee:", fee);
uint256 input = initialBuyAmount - fee;
uint256 virtualTokenReserve = bondingAllocation *
VIRTUAL_TOKEN_MULTIPLIER_BPS / 10000;
uint256 tokenBalance = ERC20(token).balanceOf(creator);
virtualTokenReserve = virtualTokenReserve - tokenBalance;

uint256 virtualEthReserve = 1 ether + input;
uint256 realEthReserve = input;
uint256 realTokenReserve = bondingAllocation - tokenBalance;

assertEq(bondingCurve.virtualEthReserve(), virtualEthReserve); //
Check virtual ETH reserve
assertEq(bondingCurve.realEthReserve(), realEthReserve); // Check
real ETH reserve
assertEq(bondingCurve.virtualTokenReserve(), virtualTokenReserve); //
Check virtual token reserve
assertEq(bondingCurve.realTokenReserve(), realTokenReserve); // Check
real token reserve

// Check fee distribution
assertEq(bondingCurve.accumulatedBuyFees(), fee); // Check
accumulated buy fees
assertEq(bondingCurve.accumulatedSellFees(), 0); // Check accumulated
sell fees
assertEq(bondingCurve.creatorAccumulatedFees(), fee / 2);
assertEq(bondingCurve.platformAccumulatedFees(), fee - fee / 2);

uint256 accumulatedBuyFees = fee;
uint256 accumulatedSellFees = 0;

// alice buy tokens and transfer to bob and bob sell tokens
uint256 aliceInitialEth = 1 ether;
vm.deal(alice, aliceInitialEth);
uint256 buyAmount = 0.2 ether;
uint256 expectOutput;
(expectOutput, fee) = bondingCurve.getBuyPrice(buyAmount);
console.log("Alice Buy Fee:", fee);
vm.prank(alice);
bondingCurve.buyTokens{value: buyAmount}(expectOutput * 95 / 100); //
Allow 5% slippage
uint256 aliceTokenBalance = ERC20(token).balanceOf(alice);
assertEq(aliceTokenBalance, expectOutput);
accumulatedBuyFees += fee;
assertEq(bondingCurve.accumulatedBuyFees(), accumulatedBuyFees);

```

```

    virtualEthReserve += buyAmount - fee;
    realEthReserve += buyAmount - fee;
    assertEq(bondingCurve.virtualEthReserve(), virtualEthReserve);
    assertEq(bondingCurve.realEthReserve(), realEthReserve);

    virtualTokenReserve -= expectOutput;
    realTokenReserve -= expectOutput;
    assertEq(bondingCurve.virtualTokenReserve(), virtualTokenReserve);
    assertEq(bondingCurve.realTokenReserve(), realTokenReserve);

    vm.prank(alice);
    ERC20(token).transfer(bob, aliceTokenBalance);
    uint256 bobTokenBalance = ERC20(token).balanceOf(bob);
    assertEq(bobTokenBalance, aliceTokenBalance);

    vm.prank(bob);
    ERC20(token).approve(address(bondingCurve), bobTokenBalance);
    (expectOutput, fee) = bondingCurve.getSellPrice(bobTokenBalance);
    accumulatedSellFees += fee;
    console.log("Bob Sell Fee:", fee);

    vm.prank(bob);
    bondingCurve.sellTokens(bobTokenBalance, expectOutput * 95 / 100);
    assertEq(bondingCurve.accumulatedSellFees(), accumulatedSellFees);

    virtualEthReserve -= expectOutput + fee;
    realEthReserve -= expectOutput + fee;
    assertEq(bondingCurve.virtualEthReserve(), virtualEthReserve);
    assertEq(bondingCurve.realEthReserve(), realEthReserve);

    virtualTokenReserve += bobTokenBalance;
    realTokenReserve += bobTokenBalance;
    assertEq(bondingCurve.virtualTokenReserve(), virtualTokenReserve);
    assertEq(bondingCurve.realTokenReserve(), realTokenReserve);

    uint256 bobFinalEth = bob.balance;
    assertEq(bobFinalEth, expectOutput);
    uint256 totalFees = accumulatedBuyFees + accumulatedSellFees;
    console.log("Total Fees:", totalFees);
    assertEq(bondingCurve.creatorAccumulatedFees() +
bondingCurve.platformAccumulatedFees(), totalFees);
}

function testGraduateOnPancake() public {
    console.log("block.number:", block.number);
    vm.deal(creator, 1 ether);
    uint256 initialBuyAmount = 0.1 ether;
    vm.prank(creator);
    (address token, address bonding) =
factory.deployBondingCurveSystemAdvanced{value: initialBuyAmount +
factoryFees}(
    "TestToken",
    "TTK",
    initialBuyAmount,

```

```

        address(graduator),
        1 ether,    // virtualEth
        0          // bondingTarget has no meaning
    );
    BondingCurveV4 bondingCurve = BondingCurveV4(payable(bonding));
    // alice buy tokens and emit graduation event
    uint256 aliceInitialEth = 2.5 ether;
    vm.deal(alice, aliceInitialEth);
    vm.prank(alice);
    bondingCurve.buyTokens{value: aliceInitialEth}(0); // Allow 5%
    slippage

    uint256 creatorInitialTokenBalance = ERC20(token).balanceOf(creator);
    address uniswapPool = bondingCurve.uniswapPool();
    assertEq(uniswapPool, address(0)); // Not graduated yet, uniswapPool
    address should be 0

    uint256 totalSupply = ERC20(token).totalSupply();
    uint256 bondingAllocation = (totalSupply * BONDING_ALLOCATION_BPS) /
    10000;

    uint256 realTokenReserve = bondingCurve.realTokenReserve();
    uint256 rate = realTokenReserve * 10000 / bondingAllocation;
    console.log("Before Graduation Rate (in bps): %s %",
    log_float(rate, 2));

    vm.prank(creator);
    bondingCurve.finalizeCurve();
    uniswapPool = bondingCurve.uniswapPool();
    console.log("Uniswap Pool Address After Graduation:", uniswapPool);

    // bob buy tokens from uniswap pool
    uint256 bobInitialEth = 1 ether;
    vm.deal(bob, bobInitialEth);
    ISwapRouter.ExactInputSingleParams memory params =
    ISwapRouter.ExactInputSingleParams({
        tokenIn: WBNB,
        tokenOut: token,
        fee: 2500,
        recipient: bob,
        deadline: block.timestamp + 15,
        amountIn: 0.5 ether,
        amountOutMinimum: 0,
        sqrtPriceLimitX96: 0
    });
    vm.startPrank(bob);
    (bool success,) = WBNB.call{value: 0.5 ether}("");
    require(success, "WBNB deposit failed");
    ERC20(WBNB).approve(PANCAKE_V3_SWAP_ROUTER, 0.5 ether);
    uint256 amountOut = router.exactInputSingle(params);
    vm.stopPrank();
    uint256 bobTokenBalance = ERC20(token).balanceOf(bob);
    assertEq(bobTokenBalance, amountOut);

    // collect fees after swap
    feeCollector.claimAndDistributeFees(token);

```

```

        (uint256 creatorAccumulated0, uint256 creatorAccumulated1) =
feeCollector.getCreatorFees(token);
        console.log("Creator Accumulated token0 Fees After Swap:",
log_float(creatorAccumulated0,18));
        console.log("Creator Accumulated token1 Fees After Swap:",
log_float(creatorAccumulated1,18));

        (uint256 platformAccumulated0, uint256 platformAccumulated1) =
feeCollector.getPlatformFees(token);
        console.log("Platform Accumulated token0 Fees After Swap:",
log_float(platformAccumulated0,18));
        console.log("Platform Accumulated token1 Fees After Swap:",
log_float(platformAccumulated1,18));

        // claim the fee;
        // should revert if not creator
        vm.expectRevert(FeeCollector.NotCreator.selector);
        feeCollector.creatorClaimFees(token);

        // creator claim fees
        vm.expectEmit(true, true, true, true);
        emit FeeCollector.CreatorFeesClaimed(token,
creator,creatorAccumulated0, creatorAccumulated1);
        vm.prank(creator);
        feeCollector.creatorClaimFees(token);
        // check balances
        uint256 creatorTokenBalanceChanged = ERC20(token).balanceOf(creator) -
creatorInitialTokenBalance;
        uint256 wbnbBalance = ERC20(WBNB).balanceOf(creator);
        uint256 amount0 = token < WBNB ? creatorAccumulated0 :
creatorAccumulated1;
        uint256 amount1 = token < WBNB ? creatorAccumulated1 :
creatorAccumulated0;
        assertEquals(creatorTokenBalanceChanged, amount0);
        assertEquals(wbnbBalance, amount1);

        // check state after claim
        (creatorAccumulated0, creatorAccumulated1) =
feeCollector.getCreatorFees(token);
        assertEquals(creatorAccumulated0, 0);
        assertEquals(creatorAccumulated1, 0);

        (platformAccumulated0, platformAccumulated1) =
feeCollector.getPlatformFees(token);
        console.log("Platform Accumulated token0 Fees After Claim:",
log_float(platformAccumulated0,18));
        console.log("Platform Accumulated token1 Fees After Claim:",
log_float(platformAccumulated1,18));

        // Check if the graduator contract has any remaining tokens, small
amounts left from adding LP
        uint256 graduatorTokenBalance =
ERC20(token).balanceOf(address(graduator));

```

```

        console.log("Bonding Curve Contract Token Balance After Graduation:",
log_float(graduatoTokenBalance,18));
        uint256 graduatorEthBalance =
ERC20(WBNB).balanceOf(address(graduator));
        console.log("Bonding Curve Contract WBNB Balance After Graduation:",
log_float(graduatorEthBalance,18));
        assert(graduatorEthBalance + graduatorTokenBalance > 0);

        uint256 graduatorBalance = address(graduator).balance;
        console.log("Bonding Curve Contract BNB Balance After Graduation:",
log_float(graduatorBalance,18));
        assertEq(graduatorBalance,0);

        vm.prank(owner);
        graduator.withdrawToken(token,0);
        vm.prank(owner);
        graduator.withdrawToken(WBNB,0);

        assertEq(ERC20(token).balanceOf(address(graduator)),0);
        assertEq(ERC20(WBNB).balanceOf(address(graduator)),0);
    }

    function log_float(uint256 value, uint8 decimals) internal pure returns
(string memory) {
        uint256 base = 10 ** decimals;
        uint256 integerPart = value / base;
        uint256 decimalPart = value % base;
        // Calculate the number of leading zeros needed for the decimal part
        uint8 decimalLen = decimals;
        uint256 temp = decimalPart;
        while (temp != 0 && decimalLen > 0) {
            temp /= 10;
            decimalLen--;
        }
        // Concatenate leading zeros
        bytes memory zeros = new bytes(decimalLen);
        for (uint8 i = 0; i < decimalLen; i++) {
            zeros[i] = bytes1("0");
        }
        return string(abi.encodePacked(vm.toString(integerPart), ".", zeros,
vm.toString(decimalPart)));
    }
}

```

2. LaunchpadThenaForkingTest.sol

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import {Test} from "forge-std/Test.sol";

```



```

import {ERC1967Proxy} from
"@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {Factory,IFactory,OwnableUpgradeable} from
"../../src/launchpad/Factory.sol";
import {TokenImplementation} from
"../../src/launchpad/TokenImplementation.sol";
import {BondingCurveV4} from "../../src/launchpad/BondingCurveV4.sol";
import {LockV2,ILock} from "../../src/launchpad/LockV2.sol";
import {ThenaGraduatorV2} from
"../../src/launchpad/graduators/ThenaGraduatorV2.sol";
import {FeeCollector} from "../../src/launchpad/FeeCollector.sol";

import {console} from "forge-std/console.sol";

interface ISwapRouter {
    struct ExactInputSingleParams {
        address tokenIn;
        address tokenOut;
        address deployer;
        address recipient;
        uint256 deadline;
        uint256 amountIn;
        uint256 amountOutMinimum;
        uint160 limitSqrtPrice;
    }

    /// @notice Swaps `amountIn` of one token for as much as possible of
    another token
    /// @param params The parameters necessary for the swap, encoded as
    `ExactInputSingleParams` in calldata
    /// @return amountOut The amount of the received token
    function exactInputSingle(ExactInputSingleParams calldata params) external
    payable returns (uint256 amountOut);
}

contract LaunchpadThenaForkingTest is Test {
    uint256 internal constant factoryFees = 0.1 ether;
    address internal constant FARMING_PLUGIN_FACTORY =
0xF807462b5ce54A89e2F5B847D8281030eAfE241B;
    address internal constant PANCAKESWAP_POSITION_MANAGER =
0x46A15B0b27311cedF172AB29E4f4766fbE7F4364;
    address internal constant THENA_POSITION_MANAGER =
0x643B68Bf3f855B8475C0A700b6D1020bfc21d02e;
    address internal constant THENASWAP_FACTORY =
0x30055F87716d3DFD0E5198C27024481099fB4A98;
    address internal constant THENA_V3_SWAP_ROUTER =
0xb85Fdbb78a735584592Df49ED7cD061b01A2e6B7;
    address internal constant WBNB =
0xbb4CdB9cBd36B01bD1cBaEBF2De08d9173bc095c;
    uint16 internal constant BONDING_ALLOCATION_BPS = 8000; // 80% for
bonding curve
    uint16 internal constant DEX_ALLOCATION_BPS = 2000; // 20% for DEX
liquidity

```



```

uint16 internal constant VIRTUAL_TOKEN_MULTIPLIER_BPS = 13500;

address internal owner = makeAddr("owner");
address internal feeTo = makeAddr("feeTo");
address internal creator = makeAddr("creator");
address internal alice = address(0x1234);
address internal bob = address(0x5678);

Factory internal factory;
LockV2 internal lock;
TokenImplementation internal token_impl;
BondingCurveV4 internal bonding_impl;
ThenaGraduatorV2 internal graduator;
FeeCollector internal feeCollector;
ISwapRouter internal router = ISwapRouter(THENA_V3_SWAP_ROUTER);

function setUp() public {
    vm.createSelectFork("your endpoint here",79555399);

    token_impl = new TokenImplementation();
    bonding_impl = new BondingCurveV4();
    Factory impl = new Factory();
    lock = new LockV2(owner, PANCAKESWAP_POSITION_MANAGER,
THENA_POSITION_MANAGER);
    IFactory.BondingCurveSettings memory defaultSettings =
IFactory.BondingCurveSettings({
        virtualEth: 20 ether,
        bondingTarget: 70 ether,
        minContribution: 0.01 ether,
        buyFee: 100, // 1%
        sellFee: 100, // 1%
        creatorFeeBps: 5000, // 50% to creator
        weth: WBNB,
        feeTo: feeTo
    });
    ERC1967Proxy proxy = new ERC1967Proxy(
        address(impl),
        abi.encodeWithSelector(
            Factory.initialize.selector,
            factoryFees,
            owner,
            token_impl,
            bonding_impl,
            lock,
            defaultSettings
        )
    );
    factory = Factory(payable(address(proxy)));

    vm.startPrank(owner);
    feeCollector = new FeeCollector(feeTo,WBNB);
    graduator = new ThenaGraduatorV2(
        address(factory),
        THENASWAP_FACTORY,

```

```

        FARMING_PLUGIN_FACTORY,
        THENA_POSITION_MANAGER
    );
    // Related settings, quite complex
    graduator.setFeeCollector(address(feeCollector));
    graduator.setLockContract(address(lock));
    factory.setGraduatorWhitelisted(address(graduator),true);
    lock.setAuthorizedFeeClaimer(address(feeCollector),true);
    feeCollector.setAuthorizedRegistrar(address(graduator),true);
    vm.stopPrank();
}

function testBuyAndSellToken() public {
    // Additional test logic can be implemented here to test the forking
    setup
    // 1. DeployBondingCurveSystem
    vm.deal(creator, 1 ether);
    uint256 initialBuyAmount = 0.1 ether;
    vm.prank(creator);
    (address token,address bonding) =
    factory.deployBondingCurveSystemAdvanced{value: initialBuyAmount +
    factoryFees}(
        "TestToken",
        "TTK",
        initialBuyAmount,
        address(graduator),
        1 ether,    // virtualEth
        1 ether    // bondingTarget
    );
    // 2. Check the results after deployment
    // Check deployment fees
    assertEq(address(factory).balance, factoryFees); // 0.1 ether
    deployment fees

    BondingCurveV4 bondingCurve = BondingCurveV4(payable(bonding));
    // Check total token supply
    uint256 totalSupply = ERC20(token).totalSupply();
    assertEq(totalSupply, 1_000_000_000 ether); // Total supply should be
    1,000,000,000 TTK

    // Check token allocation
    uint256 bondingAllocation = (totalSupply * BONDING_ALLOCATION_BPS) /
    10000;
    assertEq(bondingCurve.bondingAllocation(), bondingAllocation);
    assertEq(bondingCurve.dexAllocation(), totalSupply -
    bondingAllocation);

    // Check reserves
    uint256 fee = initialBuyAmount * 100 / 10000; // 1% buyFee
    console.log("Initial Buy Fee:", fee);
    uint256 input = initialBuyAmount - fee;
    uint256 virtualTokenReserve = bondingAllocation *
    VIRTUAL_TOKEN_MULTIPLIER_BPS / 10000;
    uint256 tokenBalance = ERC20(token).balanceOf(creator);

```

```

virtualTokenReserve = virtualTokenReserve - tokenBalance;

uint256 virtualEthReserve = 1 ether + input;
uint256 realEthReserve = input;
uint256 realTokenReserve = bondingAllocation - tokenBalance;

assertEq(bondingCurve.virtualEthReserve(), virtualEthReserve); //
Check virtual ETH reserve
assertEq(bondingCurve.realEthReserve(), realEthReserve); // Check
real ETH reserve
assertEq(bondingCurve.virtualTokenReserve(), virtualTokenReserve); //
Check virtual token reserve
assertEq(bondingCurve.realTokenReserve(), realTokenReserve); // Check
real token reserve

// Check fee distribution
assertEq(bondingCurve.accumulatedBuyFees(), fee); // Check
accumulated buy fees
assertEq(bondingCurve.accumulatedSellFees(), 0); // Check accumulated
sell fees
assertEq(bondingCurve.creatorAccumulatedFees(), fee / 2);
assertEq(bondingCurve.platformAccumulatedFees(), fee - fee / 2);

uint256 accumulatedBuyFees = fee;
uint256 accumulatedSellFees = 0;

// alice buy tokens and transfer to bob and bob sell tokens
uint256 aliceInitialEth = 1 ether;
vm.deal(alice, aliceInitialEth);
uint256 buyAmount = 0.2 ether;
uint256 expectOutput;
(expectOutput, fee) = bondingCurve.getBuyPrice(buyAmount);
console.log("Alice Buy Fee:", fee);
vm.prank(alice);
bondingCurve.buyTokens{value: buyAmount}(expectOutput * 95 / 100);
uint256 aliceTokenBalance = ERC20(token).balanceOf(alice);
assertEq(aliceTokenBalance, expectOutput);
accumulatedBuyFees += fee;
assertEq(bondingCurve.accumulatedBuyFees(), accumulatedBuyFees);

virtualEthReserve += buyAmount - fee;
realEthReserve += buyAmount - fee;
assertEq(bondingCurve.virtualEthReserve(), virtualEthReserve);
assertEq(bondingCurve.realEthReserve(), realEthReserve);

virtualTokenReserve -= expectOutput;
realTokenReserve -= expectOutput;
assertEq(bondingCurve.virtualTokenReserve(), virtualTokenReserve);
assertEq(bondingCurve.realTokenReserve(), realTokenReserve);

vm.prank(alice);
ERC20(token).transfer(bob, aliceTokenBalance);
uint256 bobTokenBalance = ERC20(token).balanceOf(bob);
assertEq(bobTokenBalance, aliceTokenBalance);

```

```

vm.prank(bob);
ERC20(token).approve(address(bondingCurve), bobTokenBalance);
(expectOutput, fee) = bondingCurve.getSellPrice(bobTokenBalance);
accumulatedSellFees += fee;
console.log("Bob Sell Fee:", fee);

vm.prank(bob);
bondingCurve.sellTokens(bobTokenBalance, expectOutput * 95 / 100);
assertEq(bondingCurve.accumulatedSellFees(), accumulatedSellFees);

virtualEthReserve -= expectOutput + fee;
realEthReserve -= expectOutput + fee;
assertEq(bondingCurve.virtualEthReserve(), virtualEthReserve);
assertEq(bondingCurve.realEthReserve(), realEthReserve);

virtualTokenReserve += bobTokenBalance;
realTokenReserve += bobTokenBalance;
assertEq(bondingCurve.virtualTokenReserve(), virtualTokenReserve);
assertEq(bondingCurve.realTokenReserve(), realTokenReserve);

uint256 bobFinalEth = bob.balance;
assertEq(bobFinalEth, expectOutput);
uint256 totalFees = accumulatedBuyFees + accumulatedSellFees;
console.log("Total Fees:", totalFees);
assertEq(bondingCurve.creatorAccumulatedFees() +
bondingCurve.platformAccumulatedFees(), totalFees);
}

function testGraduateOnThena() public {
    vm.deal(creator, 1 ether);
    uint256 initialBuyAmount = 0.1 ether;
    vm.prank(creator);
    (address token, address bonding) =
factory.deployBondingCurveSystemAdvanced{value: initialBuyAmount +
factoryFees}(
    "TestToken",
    "TTK",
    initialBuyAmount,
    address(graduator),
    1 ether, // virtualEth
    0 // bondingTarget has no meaning
);
BondingCurveV4 bondingCurve = BondingCurveV4(payable(bonding));
// alice buy tokens and emit graduation event
uint256 aliceInitialEth = 2.5 ether;
vm.deal(alice, aliceInitialEth);
vm.prank(alice);
bondingCurve.buyTokens{value: aliceInitialEth}(0); // Allow 5%
slippage
uint256 creatorInitialTokenBalance = ERC20(token).balanceOf(creator);
address uniswapPool = bondingCurve.uniswapPool();
assertEq(uniswapPool, address(0)); // Not graduated yet, uniswapPool
address should be 0

```

```

uint256 totalSupply = ERC20(token).totalSupply();
uint256 bondingAllocation = (totalSupply * BONDING_ALLOCATION_BPS) /
10000;
uint256 realTokenReserve = bondingCurve.realTokenReserve();
uint256 rate = realTokenReserve * 10000 / bondingAllocation;
console.log("Before Graduation Rate (in bps): %s %",
log_float(rate,2));

vm.prank(creator);
bondingCurve.finalizeCurve();
uniswapPool = bondingCurve.uniswapPool();
console.log("Uniswap Pool Address After Graduation:", uniswapPool);

// bob buy tokens from uniswap pool
uint256 bobInitialEth = 1 ether;
vm.deal(bob, bobInitialEth);
ISwapRouter.ExactInputSingleParams memory params =
ISwapRouter.ExactInputSingleParams({
    tokenIn: WBNB,
    tokenOut: token,
    deployer: FARMING_PLUGIN_FACTORY,
    recipient: bob,
    deadline: block.timestamp + 15,
    amountIn: 0.5 ether,
    amountOutMinimum: 0,
    limitSqrtPrice: 0
});
vm.startPrank(bob);
(bool success,) = WBNB.call{value: 0.5 ether}("");
require(success, "WBNB deposit failed");
ERC20(WBNB).approve(THENA_V3_SWAP_ROUTER, 0.5 ether);
uint256 amountOut = router.exactInputSingle(params);
vm.stopPrank();
uint256 bobTokenBalance = ERC20(token).balanceOf(bob);
assertEq(bobTokenBalance, amountOut);

// collect fees after swap
feeCollector.claimAndDistributeFees(token);
(uint256 creatorAccumulated0, uint256 creatorAccumulated1) =
feeCollector.getCreatorFees(token);
console.log("Creator Accumulated token0 Fees After Swap:",
log_float(creatorAccumulated0,18));
console.log("Creator Accumulated token1 Fees After Swap:",
log_float(creatorAccumulated1,18));

(uint256 platformAccumulated0, uint256 platformAccumulated1) =
feeCollector.getPlatformFees(token);
console.log("Platform Accumulated token0 Fees After Swap:",
log_float(platformAccumulated0,18));
console.log("Platform Accumulated token1 Fees After Swap:",
log_float(platformAccumulated1,18));

// claim the fee should revert if not creator

```

```

vm.expectRevert(FeeCollector.NotCreator.selector);
feeCollector.creatorClaimFees(token);

// creator claim fees
if(creatorAccumulated0>0 || creatorAccumulated1 > 0) {
    vm.expectEmit(true, true, true, true);
    emit FeeCollector.CreatorFeesClaimed(token,
creator,creatorAccumulated0, creatorAccumulated1);
    vm.prank(creator);
    feeCollector.creatorClaimFees(token);
}

// check balances
uint256 creatorTokenBalanceChanged = ERC20(token).balanceOf(creator) -
creatorInitialTokenBalance;
uint256 wbnbBalance = ERC20(WBNB).balanceOf(creator);
uint256 amount0 = token < WBNB ? creatorAccumulated0 :
creatorAccumulated1;
uint256 amount1 = token < WBNB ? creatorAccumulated1 :
creatorAccumulated0;
assertEq(creatorTokenBalanceChanged, amount0);
assertEq(wbnbBalance, amount1);

// check state after claim
(creatorAccumulated0, creatorAccumulated1) =
feeCollector.getCreatorFees(token);
assertEq(creatorAccumulated0, 0);
assertEq(creatorAccumulated1, 0);

(platformAccumulated0, platformAccumulated1) =
feeCollector.getPlatformFees(token);
console.log("Platform Accumulated token0 Fees After Claim:",
log_float(platformAccumulated0,18));
console.log("Platform Accumulated token1 Fees After Claim:",
log_float(platformAccumulated1,18));
}

function log_float(uint256 value, uint8 decimals) internal pure returns
(string memory) {
    uint256 base = 10 ** decimals;
    uint256 integerPart = value / base;
    uint256 decimalPart = value % base;
    uint8 decimalLen = decimals;
    uint256 temp = decimalPart;
    while (temp != 0 && decimalLen > 0) {
        temp /= 10;
        decimalLen--;
    }
    bytes memory zeros = new bytes(decimalLen);
    for (uint8 i = 0; i < decimalLen; i++) {
        zeros[i] = bytes1("0");
    }
}

```

```

        return string(abi.encodePacked(vm.toString(integerPart), ".", zeros,
vm.toString(decimalPart)));
    }
}

```

3. TokenImplementationTest.sol

```

// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;

import {Test} from "forge-std/Test.sol";
import {ERC1967Proxy} from
"@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {TokenImplementation} from
"../../src/launchpad/TokenImplementation.sol";

contract TokenImplementationTest is Test {
    TokenImplementation internal instance;

    function setUp() public {
        // Setup code for TokenImplementation tests
        TokenImplementation impl = new TokenImplementation();
        ERC1967Proxy proxy = new ERC1967Proxy(
            address(impl),
            abi.encodeWithSelector(
                TokenImplementation.initialize.selector,
                "Test Token",
                "TTK"
            )
        );
        instance = TokenImplementation(address(proxy));
    }

    function testInitializeRepeatShouldBeRevert() public {
        vm.expectRevert("InvalidInitialization()");
        instance.initialize("Another Token", "ATK");
    }

    function testMintTotalSupplySuccess() public {
        address recipient = address(0x123);
        vm.expectEmit(true, true, true, true);
        emit TokenImplementation.InitialMintCompleted(recipient,
instance.TOTAL_SUPPLY());
        instance.mintTotalSupply(recipient);
        uint256 balance = instance.balanceOf(recipient);
        assertEq(balance, instance.TOTAL_SUPPLY());
    }

    function testMintTotalSupplyOnlyFactoryShouldRevert() public {

```

```

        address recipient = address(0x123);
        vm.prank(address(0x456)); // Change msg.sender to a non-factory
address
        vm.expectRevert(TokenImplementation.OnlyFactory.selector);
        instance.mintTotalSupply(recipient);
    }

    function testMintTotalSupplyAlreadyMintedShouldRevert() public {
        assert(!instance.isMinted());
        address recipient = address(0x123);
        instance.mintTotalSupply(recipient);
        assert(instance.isMinted());
        vm.expectRevert(TokenImplementation.AlreadyMinted.selector);
        instance.mintTotalSupply(recipient);
        assert(instance.isMinted());
    }

    function testTransferBeforeMintShouldRevert() public {
        address recipient = address(0x123);
        address sender = address(0x456);
        vm.prank(sender);
        vm.expectRevert("Transfers disabled before mint");
        instance.transfer(recipient, 0);
    }

    function testTransferAfterMintShouldSucceed() public {
        address recipient = address(0x123);
        address sender = address(0x456);
        instance.mintTotalSupply(sender);
        vm.prank(sender);
        instance.transfer(recipient, 100);
        uint256 balance = instance.balanceOf(recipient);
        assertEq(balance, 100);
    }
}

```

11.2 External Functions Check Points

1. Launchpad.md

File: src/launchpad/Factory.sol

contract: Factory is IFactory, Initializable, Ownable2StepUpgradeable, ReentrancyGuardUpgradeable

(Empty fields in the table represent things that are not required or relevant)

Index	Function	State Mutability	Modifier	Unit Test	Miscellaneous
1	initialize(uint256,address,address, address,address,BondingCurveSettings)		initializer		OnlyOnce
2	updateBondingCurveSettings(BondingCurveSettings)		onlyOwner		
3	getBondingCurveSettings()	view			
4	deployBondingCurveSystemAdvanced(string,string,uint256, address,uint256,uint256)	payable	nonReentrant		Yes
5	updateDeploymentFee(uint256)		onlyOwner		
6	withdrawFees(address)		onlyOwner		
7	getDeploymentFee()	view			
8	getTokenImplementation()	view			
9	getBondingCurveImplementation()	view			
10	getLockContract()	view			
11	getBondingCurveForToken(address)	view			
12	getTokenForBondingCurve(address)	view			
13	setGraduatorWhitelisted(address,bool)		onlyOwner		
14	setGraduatorsWhitelistedBatch(address[],bool)		onlyOwner		
15	isGraduatorWhitelisted(address)	view			
16	receive()	payable			

File: src/launchpad/FeeCollector.sol

contract: FeeCollector is Ownable2Step, ReentrancyGuard, Pausable

(Empty fields in the table represent things that are not required or relevant)

Index	Function	StateMutability	Modifier	Unit Test	Miscellaneous
1	registerGraduatedToken(address,address,address,address,uint256,address)	payable	nonReentrant		OnlyCurve
2	claimAndDistributeFees(address)		nonReentrant, whenNotPaused		
3	creatorClaimFees(address)		nonReentrant, whenNotPaused		
4	platformWithdrawFees(address)		onlyOwner, nonReentrant		
5	getCreatorFees(address)	view			
6	getPlatformFees(address)	view			
7	getTokenInfo(address)	view			
8	getAllGraduatedTokens()	view			
9	getGraduatedTokenCount()	view			
10	isTokenRegistered(address)	view			
11	setAuthorizedRegistrar(address,bool)		onlyOwner		
12	updateFeeSplit(uint256,uint256)		onlyOwner		
13	updatePlatformFeeRecipient(address)		onlyOwner		
14	pause()		onlyOwner		
15	unpause()		onlyOwner		
16	emergencyWithdraw(address,uint256)		onlyOwner		
17	emergencyWithdrawETH()		onlyOwner		
18	receive()	payable			

File: src/launchpad/TokenImplementation.sol

contract: TokenImplementation is Initializable, ERC20Upgradeable

(Empty fields in the table represent things that are not required or relevant)

Index	Function	StateMutability	Modifier	Unit Test	Miscellaneous
1	initialize(string,string)		initializer		OnlyOnce
2	mintTotalSupply(address)				OnlyOnce
3	isMinted()	view			

File: src/launchpad/BondingCurveV4.sol

contract: BondingCurveV4 is IBondingCurve, Ownable2Step, ReentrancyGuard, IERC721Receiver

(Empty fields in the table represent things that are not required or relevant)

Index	Function	StateMutability	Modifier	Unit Test	Miscellaneous
1	initialize(address,address,address, IFactory.BondingCurveSettings)				OnlyOnce
2	initializeWithGraduator(address, address,address, IFactory.BondingCurveSettings,address)				OnlyOnce
3	buyTokens(uint256)	payable	nonReentrant, onlyPhase(Phase.Bonding)		
4	buyTokensFor(address,uint256)	payable	nonReentrant, onlyPhase(Phase.Bonding)		
5	sellTokens(uint256,uint256)		nonReentrant, onlyPhase(Phase.Bonding)		
6	finalizeCurve()		onlyOwner, nonReentrant		
7	getTotalFeesCollected()	view			
8	getAccumulatedFees()	view			
9	transferAccumulatedFeesToCollector (address)				OnlyGraduator
10	getBuyPrice(uint256)	view			
11	getSellPrice(uint256)	view			
12	getCurrentPrice()	view			
13	getPricImpact(uint256,uint256)	view			
14	virtualEth()	view			
15	virtualTokens()	view			
16	realEth()	view			
17	realTokens()	view			
18	buyFee()	view			
19	sellFee()	view			
20	creatorFeeBps()	view			
21	tokenBalance()	view			
22	getPhase()	view			
23	getDualReserveState()	view			
24	getGraduationProgress()	view			
25	getEconomicSnapshot()	view			
26	getGraduationAmounts()	view			
27	getExpectedGraduationPrices()	view			
28	getFeeBreakdown()	view			
29	onERC721Received(address, address,uint256,bytes)	pure			
30	receive()	payable			

File: src/launchpad/LockV2.sol

contract: LockV2 is ILock, Context, ReentrancyGuard, IERC721Receiver

(Empty fields in the table represent things that are not required or relevant)

Index	Function	StateMutability	Modifier	Unit Test	Miscellaneous
1	owner()	view			
2	transferOwnership(address)		onlyOwner		
3	lockNFTWithDuration(address,uint256,address,uint256)		nonReentrant		supportedPositionManagers
4	lockNFT(uint256,address)		nonReentrant		supportedPositionManagers
5	claimFeesFor(uint256,address)		nonReentrant		authorizedFeeClaimers
6	setAuthorizedFeeClaimer(address,bool)		onlyOwner		
7	getNFTsByOwner(address)	view			
8	isNFTLocked(uint256)	view			
9	isNFTBurned(uint256)	view			
10	getRemainingLockTime(uint256)	view			
11	addPositionManager(address)		onlyOwner		
12	removePositionManager(address)		onlyOwner		
13	onERC721Received(address,address,uint256,bytes)	pure			
14	checkAvailableFees(uint256)	view			



<https://medium.com/@FairyproofT>



<https://twitter.com/FairyproofT>



<https://www.linkedin.com/company/fairyproof-tech>



https://t.me/Fairyproof_tech



Reddit: <https://www.reddit.com/user/FairyproofTech>

